

Video Recording Script — Meridian CRM (Log Interaction Screen)

Target: 10–15 minutes. Four required sections, timed below.

Everything in *italics* is a talking point. Everything in `code` is typed or clicked on screen. The chat phrases below are the exact ones verified to route to each tool — don't improvise them during the recording.

Before you hit record — setup checklist

#	Check	How
1	Postgres is running	pgAdmin, or <code>Get-Service postgresql-x64-18</code>
2	Backend is up	<code>cd backend && uvicorn app.main:app --reload --port 8000</code>
3	Frontend is up	<code>cd frontend && npm run dev</code> → http://localhost:5173
4	LLM is live	open http://localhost:8000/health → must show <code>"llm_configured": true</code>
5	One HCP exists	the DB is empty; add Dr. Ananya Mehta (Cardiology, Apollo Hospitals, Tier A, preferred products <code>Cardizem</code> , <code>Brilinta</code>) from the sidebar + Add
6	Log one interaction	so the timeline and the history tool have something to show
7	Close other tabs	the browser will ask for microphone permission during the voice-note demo

If `llm_configured` is false, stop and fix it. The app will still work, but it silently falls back to heuristics and your extraction will look dumb on camera.

Have pgAdmin open in a second tab — you'll show the tables at the end.

Part 1 — What I understood from the task (0:00 – 1:30)

"The brief was to build the Log Interaction screen for an AI-first pharma CRM — the one thing a field rep does over and over after every visit to a doctor.

The real problem isn't 'add a form'. Reps don't fill in forms. They finish a meeting, they're in the car park, and the last thing they want is twelve fields. So the interaction never gets logged, or it gets logged badly a week later. That's the friction the task is really about.

So the point of the AI isn't decoration — it's to remove the form-filling without removing the structure the business needs. The rep says one sentence in plain English; the agent turns it into a structured,

queryable CRM record.

*The one principle I held onto throughout: **the AI fills, the human confirms**. The agent never gets the last word. Everything it extracts lands in the form, highlighted, where the rep can see it and correct it. A hand edit always beats the model. In a regulated industry you cannot have a language model quietly writing records nobody checked."*

Part 2 — Frontend walkthrough (1:30 – 5:30)

The screen

"Left is the structured form. Right is the AI assistant. They're side by side on purpose — they're not two modes, they're one workspace sharing one record."

Point out the header badge: `llama-3.3-70b-versatile` and `LangGraph` .

Way 1 — the chat fills the form (the core demo)

Select **Dr. Ananya Mehta**. Type into the assistant:

Today I met with Dr. Mehta and discussed Cardizem efficacy. The sentiment was positive, and I shared the brochures. Left 5 samples. Wants a follow-up next month.

Pause. Let the form fill on camera. Then narrate what landed:

- HCP Name, Date, Time
- Topics Discussed — *a written summary, not an echo of what I typed*
- Materials Shared → `brochures` chip
- Samples Distributed → `Cardizem: 5`
- Sentiment → **Positive** selected
- Outcomes and Follow-up Actions — *the model inferred these; I never said the word "outcome"*

"Notice the highlighted fields — that's the app telling me 'the AI touched these, check them.'"

Read the assistant's reply out loud — it confirms, names the fields it filled, and **proactively offers to schedule the follow-up**.

The subtle bit — worth calling out

*"The agent already saved this record. So watch the button — it says **Update interaction**, not Log. The form is bound to the row that exists. If I confirm, it PATCHes; it does not create a second copy. Getting that wrong would silently duplicate every AI-logged interaction."*

Change Sentiment to **Neutral** by hand → click **Update interaction** → show the timeline updates, and the count does **not** go up.

Way 2 — the voice note (consent gate)

Click 🗨 **Summarize from Voice Note (Requires Consent)**.

"It doesn't start recording. It asks first."

"This is a compliance gate, not a checkbox. Nothing is captured until I confirm the HCP agreed, and that answer is stored on the record as `consent_obtained` — so the interaction carries its own audit trail. In pharma, you can't record a doctor because a form felt convenient."

Click **Consent given**, dictate:

"Quick call with Dr. Mehta about Brilinta dosing, she was receptive, left two samples."

Click **Stop & summarize** → the form fills from your voice.

Way 3 — the form itself

"And of course I can just type. Every field is editable, and anything I set by hand wins over the model."

Show a chip: type in Attendees, press Enter, click the x.

Part 3 — All 5 LangGraph tools (5:30 – 10:00)

"The agent has five tools. The router picks exactly one per message. Let me trigger each."

Keep the browser devtools **Network** tab open on `/api/v1/chat` — after each message, expand the response and point at `"tool_used"`. That is your proof, not vibes.

1. `log_interaction` — captures the interaction

Log a visit with Dr. Mehta, discussed Cardizem dosing, very positive, left 5 samples

→ `tool_used: "log_interaction"` · a new row appears in the timeline · the form fills.

2. `edit_interaction` — modifies a logged interaction

Actually, change the sentiment to negative and note that we discussed formulary access

→ `tool_used: "edit_interaction"`

"I didn't give it an ID. It edited the most recent interaction — that's the natural 'actually, change that' flow. And the patch is validated against an allow-list before it's applied, so the model can't write to a field it shouldn't."

3. `get_interaction_history` — grounds it in the relationship

What did we discuss recently?

→ `tool_used: "get_interaction_history"` — the agent summarises past interactions.

4. `get_hcp_details` — resolves the doctor

Tell me about this doctor – what's her specialty and preferred products?

→ `tool_used: "get_hcp_details"` — returns specialty, institution, tier, preferred products.

5. `schedule_followup` — the next best action

Schedule a follow-up in 2 weeks to share the outcomes data

→ `tool_used: "schedule_followup"` — the **Follow-ups panel** updates with a due date.

Close the section

*"Five tools, one hop each. And the important design point: **both** the form and the chat call the same tools. The AI logic lives in exactly one file — `agent/tools.py`. A form submit calls `Log_interaction` directly; a chat message goes through the full graph. There's no second implementation to drift."*

Part 4 — Code & project structure (10:00 – 13:30)

Open the repo in the editor. Keep it to the shape, not line-by-line.

The stack

React 18 + Redux Toolkit → FastAPI (async) → LangGraph → Groq → SQLAlchemy → **PostgreSQL**.

Backend — `backend/app/`

<code>main.py</code>	FastAPI app; lifespan creates tables + compiles the graph ONCE
<code>config.py</code>	pydantic-settings – the model is CONFIG, never hardcoded
<code>models.py</code>	SQLAlchemy ORM: HCP, Interaction, FollowUp
<code>schemas.py</code>	Pydantic request/response models
<code>agent/</code>	
<code>graph.py</code>	the LangGraph StateGraph
<code>tools.py</code>	the 5 tools ← the heart of the app
<code>prompts.py</code>	system prompts (JSON-mode)
<code>api/routes/</code>	health · hcps · interactions · followups · chat · stats
<code>llm/client.py</code>	Groq via the OpenAI SDK + the offline fallback

Show `agent/graph.py` — the graph is four nodes:

```
START → load_context → router → execute_tool → responder → END
```

"`load_context` loads the HCP's profile and last few interactions first, so the agent reasons **with** the relationship instead of blind. `router` asks the LLM, in JSON mode, which one tool to run. `execute_tool` runs it in a DB transaction. `responder` writes the reply."

Show `agent/tools.py` → `log_interaction`: raw notes → LLM (JSON mode) extraction → validate/normalise → **merge overrides (the rep wins)** → persist.

Frontend — `frontend/src/`

```
store/interactionsSlice.js  ← the key file: holds the DRAFT
components/
  LogInteractionScreen.jsx   form + assistant, side by side
  InteractionForm.jsx        a controlled view over the draft
  ChatPanel.jsx              on a tool result → dispatch applyDraft()
  VoiceNoteButton.jsx        consent gate + dictation
  ChipInput.jsx              attendees / materials / samples
```

"The trick that makes the whole thing work is one piece of Redux state — the **draft**. The form is just a controlled view over it. Both the rep typing and the agent extracting write to that same draft. That's the entire mechanism behind 'the AI fills the form'."

Two engineering decisions worth defending

1. Why JSON mode and not native tool-calling. "`gemma2` doesn't reliably support Groq's `tools=` function-calling parameter, so I don't use `bind_tools`. The router uses JSON mode and the graph executes the chosen tool deterministically. That turned out to matter — see the next point."

2. The model in the brief no longer exists. "`gemma2-9b-it` has been **decommissioned by Groq** — every call returns a 400 `model_decommissioned`. I switched to `LLama-3.3-70b-versatile`, which the brief itself names as the alternative."

Two things made that painless. The model was never hardcoded — it's two lines of `.env`, zero code changes. And a failed LLM call degrades to a deterministic heuristic extractor instead of raising, so the CRM kept logging interactions correctly even against a dead model. The 400 went to my logs, not to the rep's screen."

The database

Switch to pgAdmin → `crm` → Schemas → public → Tables → `hcps`, `interactions`, `follow_ups`.

"No hand-written SQL. The schema is defined once in `models.py` and created from the ORM on startup, so the database can't drift from the code."

Open `interactions` → show the columns: `attendees`, `materials_shared` (kept deliberately separate from `samples_dropped`), `topics_discussed`, `outcomes`, `follow_up_actions`, `consent_obtained`.

Part 5 — Close (13:30 – 15:00)

"To summarise what I took from the task:

The job was to remove form-filling friction without losing structure. The answer was an agent that sits beside the form rather than replacing it — the rep speaks or types one sentence, the agent extracts the record, and the human confirms it. The AI fills; the human decides.

Everything funnels through the same five LangGraph tools, so there's one place where the AI logic lives. And it's built to survive the real world: if Groq is down, rate-limited, or the model gets retired underneath me — which actually happened — the rep still gets their interaction logged."

Optional, if you have 20 seconds spare — it lands well:

"One thing I'd add with more time: Alembic for versioned migrations. Right now the schema is created from the models with an additive column migration on startup, which is fine for this scope, but a production deployment needs rollbacks and a migration history."

Do-not-forget list

- ☐ `"llm_configured": true` **before** you record
- ☐ Show `tool_used` in the Network tab for **all five** tools
- ☐ Show the **consent prompt** appearing *before* any recording starts
- ☐ Show the button saying **Update interaction** (the no-duplicate design)
- ☐ Show **pgAdmin** — it proves Postgres, not SQLite
- ☐ Say the **gemma2 decommissioning** out loud — it's a strength, not an excuse
- ☐ Don't show your **Groq API key** on screen (it's in `backend/.env`)